

INF 2214

Systems Analytics Using Apache Flink and Kafka

Group 9



Presented by

Hanqi Yao

Xiaoen Hu

Yihang Liang

Kaylee Li

Content

1 Use Case & Motivation



2 System Design & Pipeline



3 Time Semantics



4 Late Data Handling



5 Demo & Reflection



Part 1.

Use Case & Motivation



Use Case: Ride-Sharing

Problem:

- Ride events arrive continuously but often out-of-order and delayed
- Revenue patterns vary across regions, time, and external conditions
- Traditional batch processing cannot accurately capture real-time system state

Objective:

- Enable real-time revenue monitoring by region
- Identify high-revenue zones and revenue trends
- Ensure correct analytics using event-time, even with delayed data



Motivation: Why Analytics?

Key Values:

- Supports dynamic pricing and regional optimization
- Enables live monitoring of raw events and aggregated insights
- Improves operational awareness and decision-making

Why Streaming?

- Ride events must be processed as they arrive
- Insights are needed in near real time, not after delays
- Event-time processing ensures correctness despite late data
- Enables continuous analytics and live dashboard updates

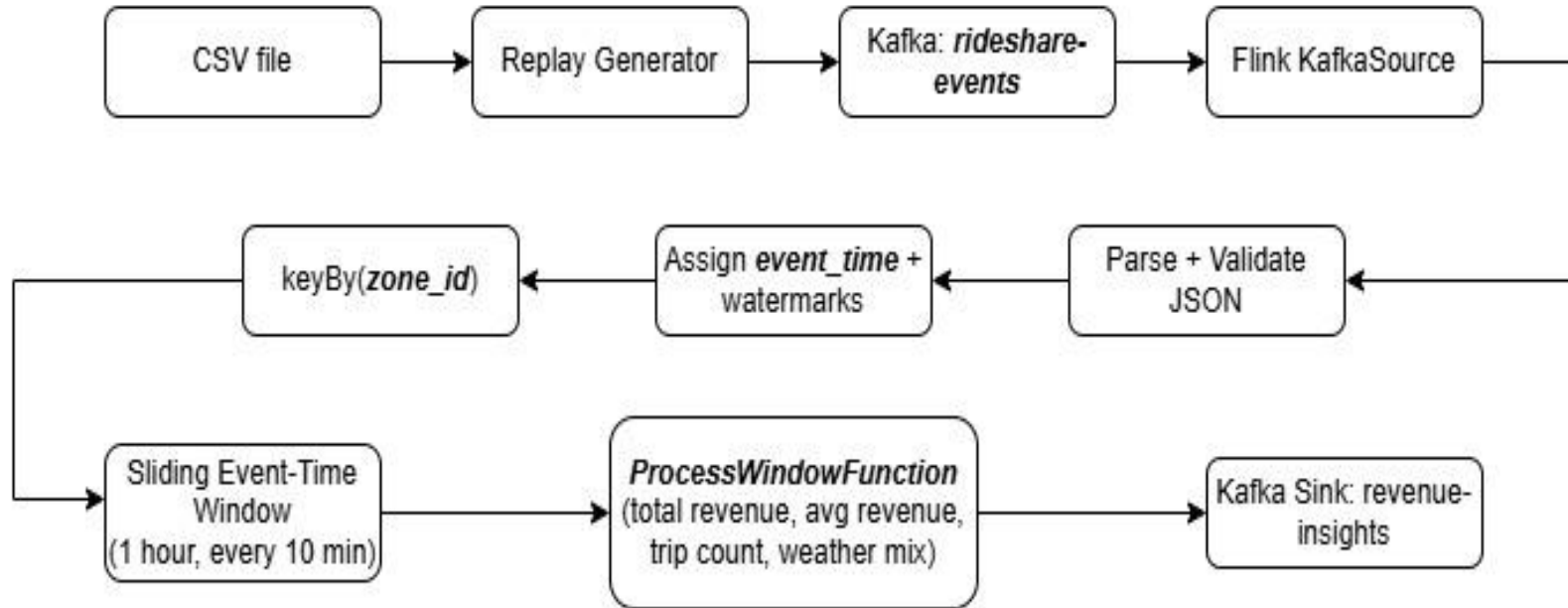
Part 2.

System Design & Pipeline

System Design Overview

- Goal: real-time rideshare revenue monitoring by region
- Input events contain both *event_time* and *ingest_time*
- CSV replay generator publishes events to Kafka in **ingest-time** order
- PyFlink processes the stream using event-time semantics
- Dashboard displays raw events, aggregated insights, and late events
- Main Kafka topics:
 - *rideshare-events* for raw input
 - *revenue-insights* for aggregated output
 - *late-rideshare-events* for too-late records

Flink Pipeline and Design Choices



Part 3.

Time Semantics

Time Semantics

Why Event Time?

- We choose **Event Time** instead of Processing Time
- Each ride event contains both **event_time** and **ingest_time**.
- Our stream may contain **out-of-order** and **delayed** records.
- **Event Time** ensures each ride is assigned to the **correct sliding window**.

Ride happens → event_time

Record arrives later → ingest_time

Flink uses event_time for windowing

Part 4.

Late Data Handling

Late Data Handling

Why Late Data Happens

- Network delays & mobile device buffering
- Ingestion latency during peak periods
- Events arrive out-of-order vs actual event time

Trade-off: Latency vs Completeness

- Priority: Low-latency output for real-time analytics.
- Accept partial incompleteness
- Suitable for ride-sharing decisions (pricing, monitoring)

1. Watermark	2. Allowed Lateness	3. Side-Outputs Stream
1-Minute Threshold	30-Second Grace Period	Beyond Accepted Lateness Thresholds
Captures the majority of out-of-order data based on observed delay ranges	Allows small late updates after window closure	Redirects extremely late events to a separate stream for monitoring
Benefit: Prevents system slowing down	Benefit: Maintains stable results for real-time dashboards	Benefit: Keeps main aggregation clean and auditable

Part 5.

Demo & Reflection

Demo & Reflection

What worked well

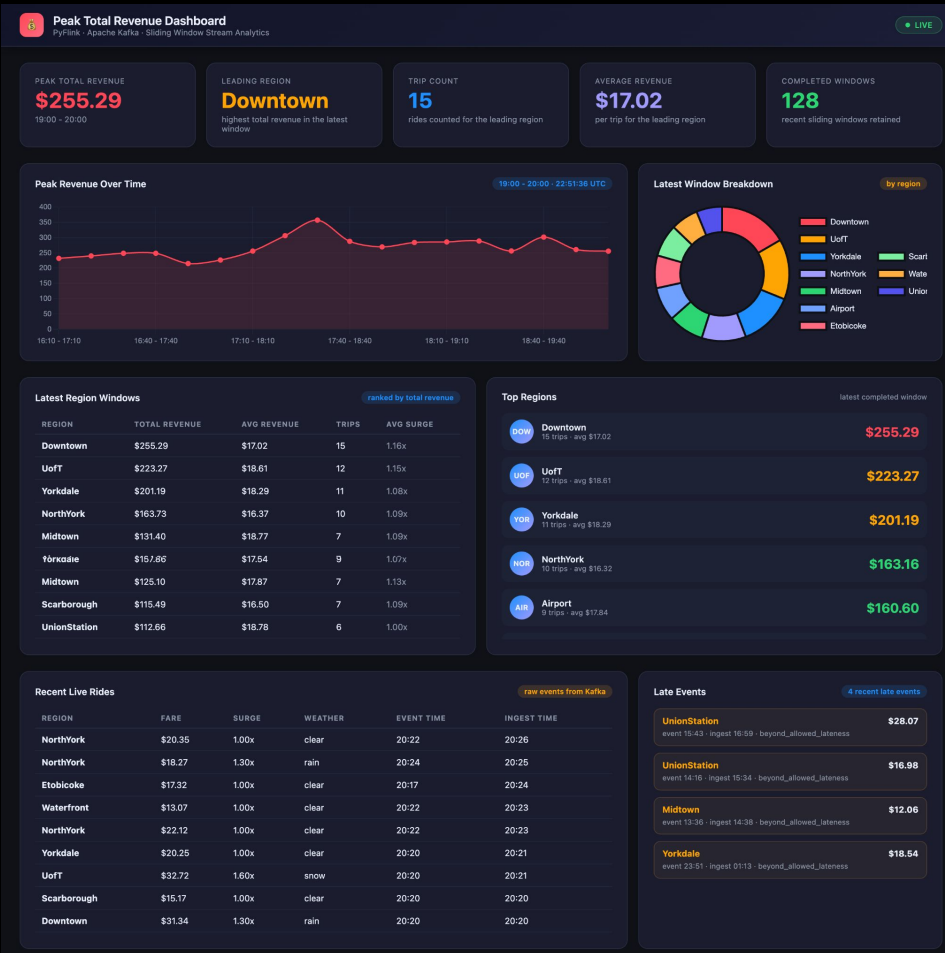
- The dashboard clearly shows **windowed revenue results, raw ride events, and late events** in one place.

One limitation

- Our dataset is **synthetic**, so it does not fully represent real-world rideshare complexity.

Future improvement

- With more time, more realistic data, test more late and out-of-order scenarios, and evaluate the pipeline under larger streaming workloads



Thank you!